

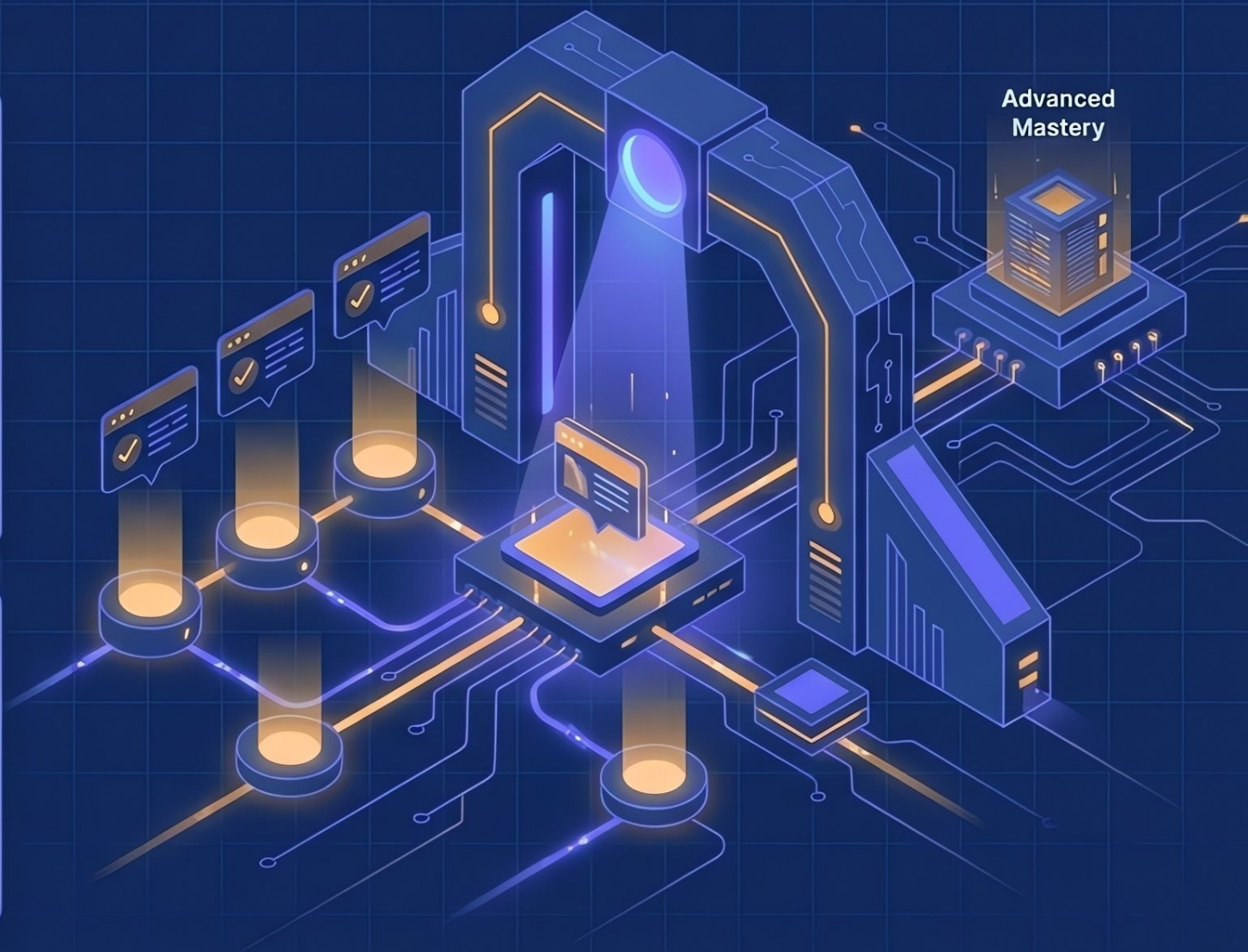
Milestone 1: Đánh giá Năng lực Thực hành Part 1 & 2

✓ Yêu cầu Tiên quyết Bắt buộc (Status: Verified)

- Đã hoàn thành 100% Video Part 1 (10 videos - Basic E-Commerce). ☒
- Đã hoàn thành chuỗi Video Part 2 (10 videos - Blazor In-depth). ☒

Mục tiêu Chuyên sâu (Under-the-hood)

- Không học cú pháp cơ bản. Tập trung **mở xẻ cơ chế vận hành nội tại** của hệ thống Blazor.



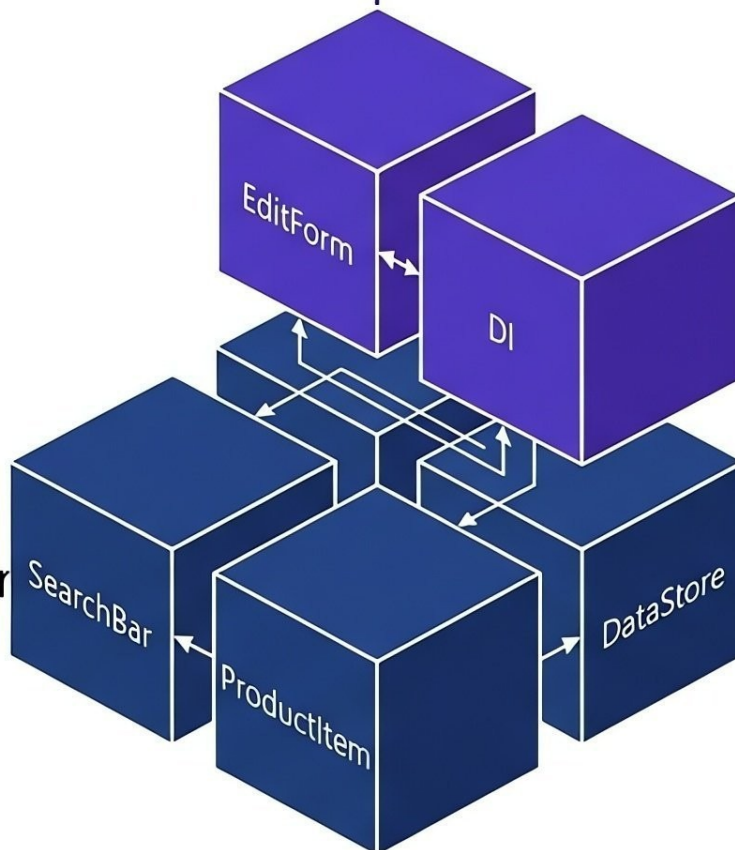
Bảng Đối soát Tiên quyết: Những gì Sinh viên ĐÃ Code Xong

Sản phẩm Part 1 (Basic E-Commerce)

- ✓ Trang chủ eShop, thanh tìm kiếm SearchBarComponent, danh sách ProductItemComponent.
- ✓ Kho dữ liệu tạm thời HardCodedProductRepository và trang ViewProductDetails.

Sản phẩm Part 2 (Blazor In-Depth)

- ✓ Giao tiếp linh kiện bằng [Parameter] và EventCallback.
- ✓ Biểu mẫu EditForm có kiểm thực dữ liệu DataAnnotations và đăng ký Service DI.



Mở xẻ Vòng đời Component: 3 Giai đoạn Cốt tử

Khởi tạo
(OnInitializedAsync)



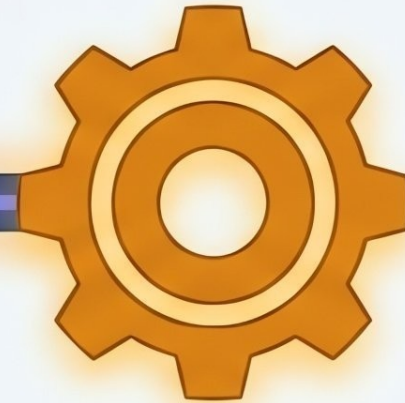
Chạy duy nhất 1 lần khi component được tạo. Nơi lý tưởng để gọi UseCase lấy dữ liệu ban đầu.

Cập nhật Tham số
(OnParametersSetAsync)



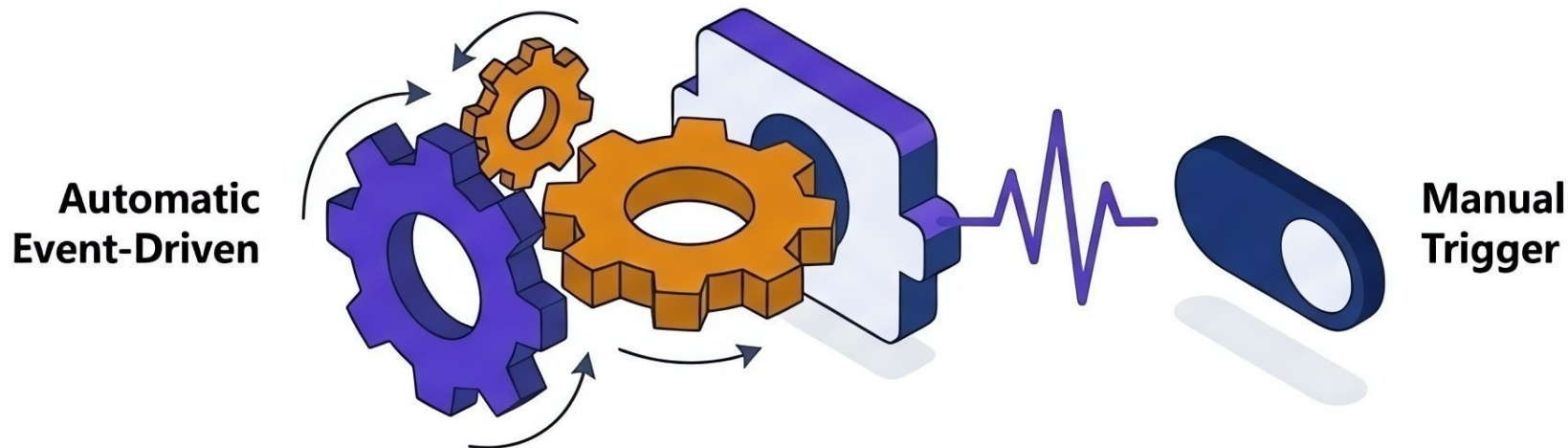
Kích hoạt mỗi khi component cha truyền giá trị [Parameter] mới vào.

Sau khi Hiển thị
(OnAfterRenderAsync)



Chạy sau khi HTML đã render lên trình duyệt. Nơi an toàn duy nhất để gọi JavaScript Interop.

Cơ chế Re-rendering: Khi nào `StateHasChanged()` Thực sự Cần?



Tự động Re-render (Automatic)

Hệ thống tự động kích hoạt sau các sự kiện UI chuẩn (@onclick, @oninput, EventCallback).

Gọi Thủ công `StateHasChanged()`

- Khi nhận dữ liệu từ một tiến trình chạy ngầm (Background Thread, Timer).
- Khi lắng nghe sự kiện từ Service bên ngoài (ví dụ: `IShoppingCart.OnChange`).



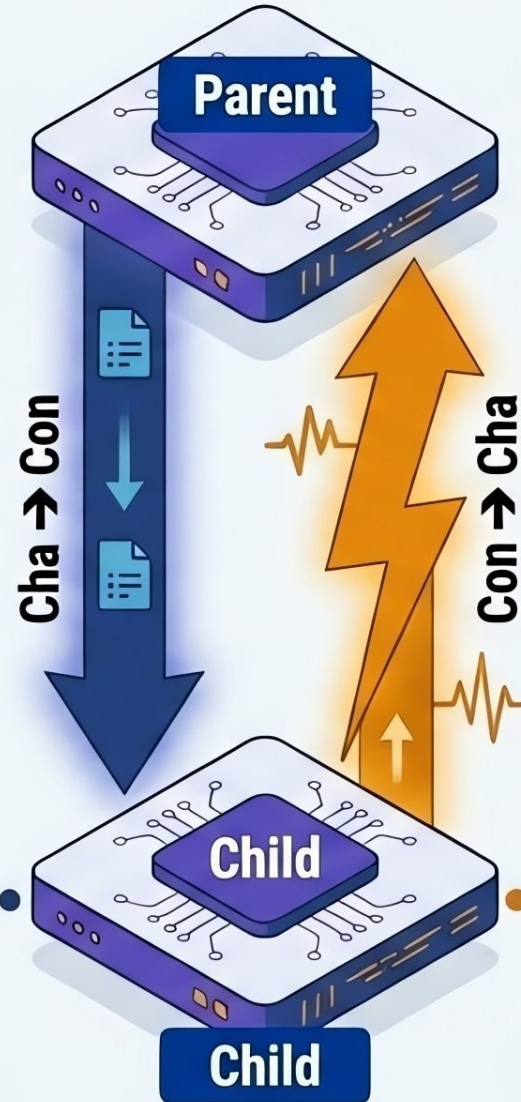
Cảnh báo Kiến trúc: Lạm dụng gọi `StateHasChanged()` tùy tiện trong `OnAfterRender` sẽ gây ra lỗi vòng lặp vô tận (Infinite Loop).

Giao tiếp Cha → Con ([Parameter]) & Con → Cha (EventCallback)

↓ **Luồng Chiều xuống**
(Cha → Con)

Công cụ: Thuộc tính [Parameter].

Cơ chế: Cha truyền dữ liệu
(`<ProductItem Product='prod' />`),
Con nhận đối tượng trạng thái.



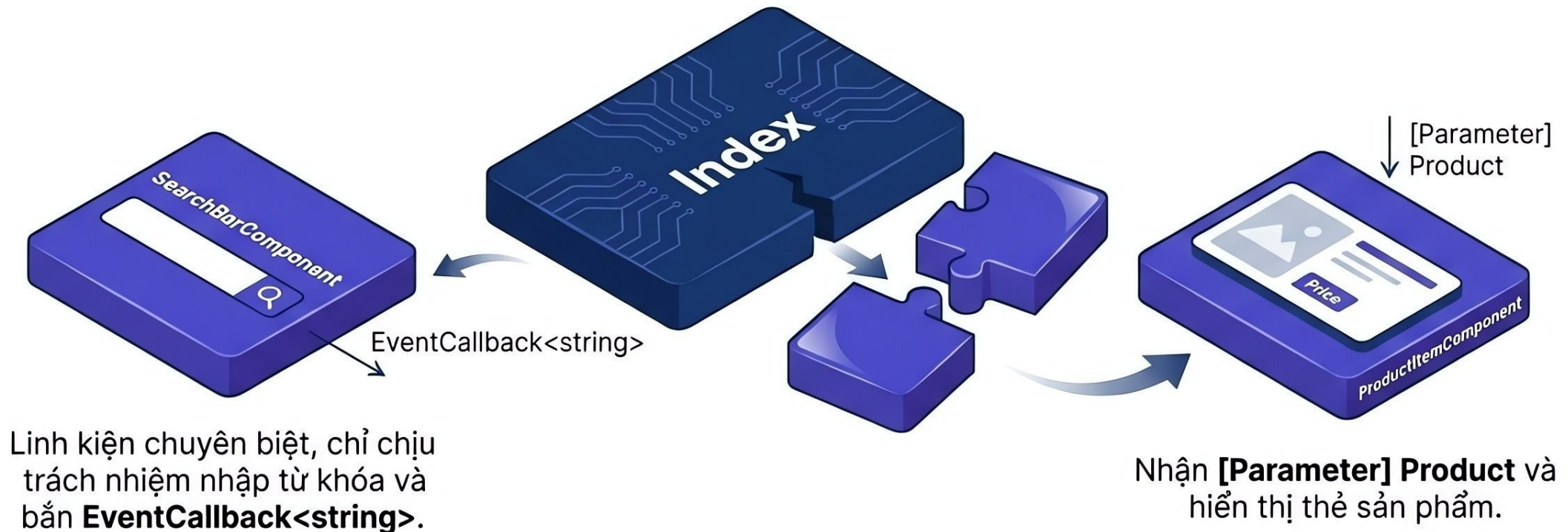
⚡ **Luồng Chiều lên**
(Con → Cha)

Công cụ: EventCallback<T>.

Cơ chế: Con phát tín hiệu
(`await OnSearch.InvokeAsync()`),
Cha lắng nghe và tự động kích
hoạt Re-render dữ liệu.

Chia tách Component Tái sử dụng: SearchBar & ProductItem

Vấn đề Kiến trúc: Tránh nhồi nhét toàn bộ mã HTML vào một trang Index.razor khổng lồ.



Lợi ích Cốt lõi: Linh kiện con hoàn toàn độc lập, dễ dàng tái sử dụng ở Trang chủ, Trang tìm kiếm, hoặc Danh mục mà không cần viết lại mã.

Truyền Dữ liệu Xuyên tầng UI với CascadingParameter

Vấn đề (Props Drilling):

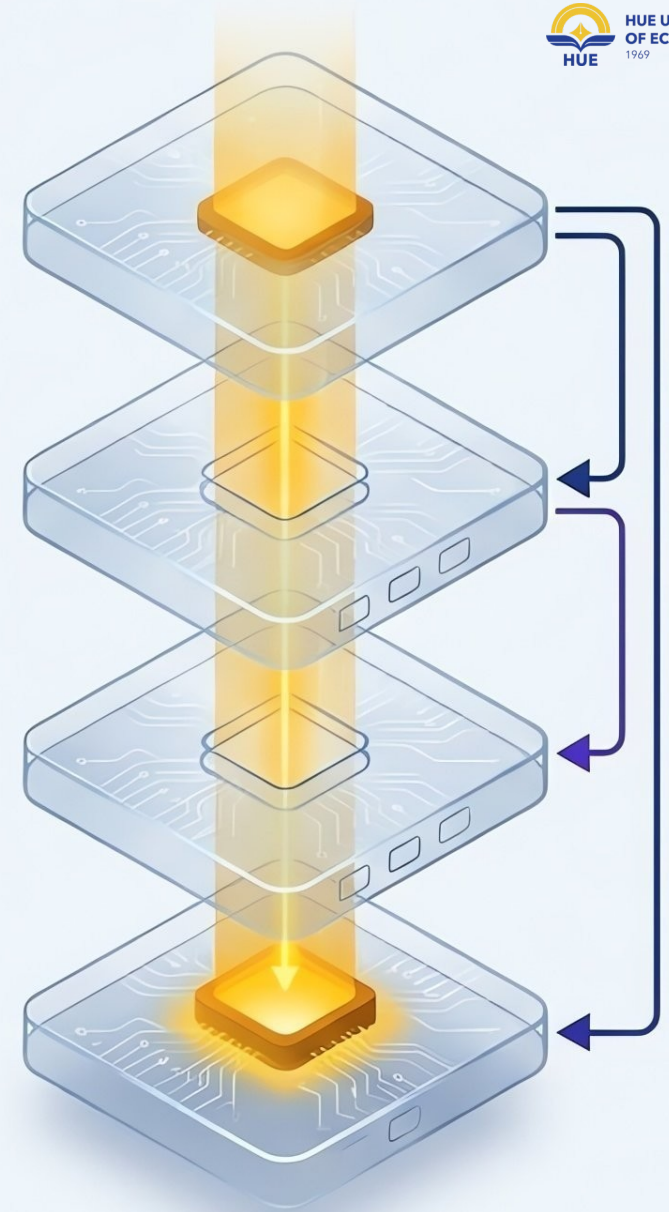
Dùng [Parameter] thông thường bắt buộc phải truyền tham số qua 4-5 tầng component trung gian dù tầng giữa không hề sử dụng đến.

Tầng Gốc

Cung cấp dữ liệu toàn cục qua thẻ `<CascadingValue Value='theme'>`.

Tầng Đích

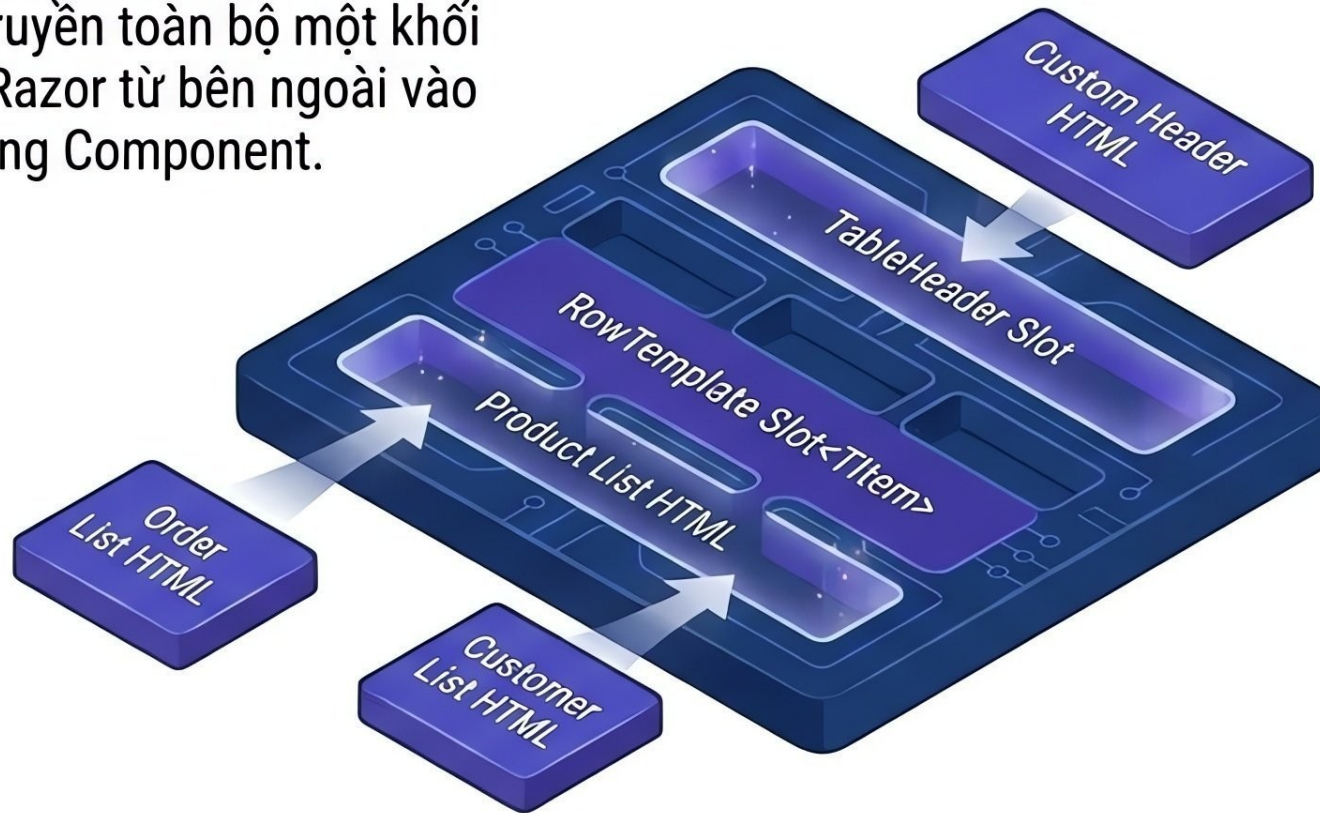
Bất kỳ Component con ở độ sâu nào cũng hứng được dữ liệu trực tiếp bằng `[CascadingParameter]`.



RenderFragment & Templated Components Doanh nghiệp

Cơ chế RenderFragment

Cho phép truyền toàn bộ một khối mã HTML/Razor từ bên ngoài vào sâu bên trong Component.



Xây dựng Bảng Dữ liệu Tổng quát (GenericTable<TItem>)

- Định nghĩa khe cắm tiêu đề:
[Parameter]
`public RenderFragment
TableHeader { get; set; }`
- Định nghĩa khe cắm dòng:
[Parameter]
`public RenderFragment<TItem>
RowTemplate { get; set; }`

Hiệu quả Kiến trúc: Một component bảng duy nhất có thể hiển thị linh hoạt
Danh sách Sản phẩm, Đơn hàng, hoặc Khách hàng.

Tham chiếu Component Con bằng Chỉ thị @ref

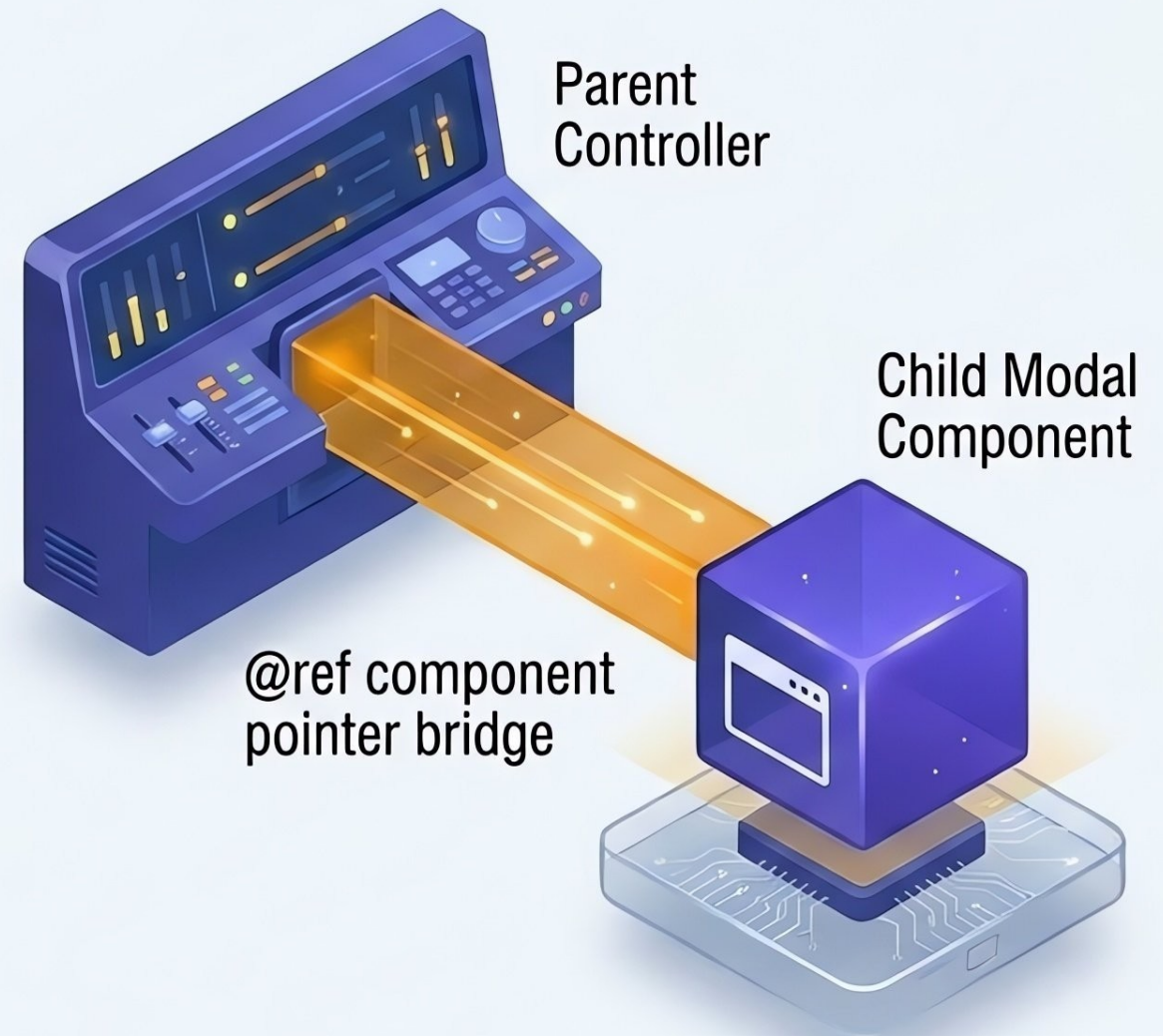
Cú pháp Tham chiếu Đoạn mã

- Gắn thẻ định danh:
`<SearchBarComponent @ref='searchBarRef' />`
- Khai báo biến nắm giữ:
`private SearchBarComponent? searchBarRef;`

Kịch bản Ứng dụng Thực tế

- Gọi trực tiếp hàm nội bộ `searchBarRef.ClearSearch()` để xóa trắng ô tìm kiếm từ bên ngoài.
- Điều khiển đóng/mở hộp thoại Popup hoặc Modal từ xa.

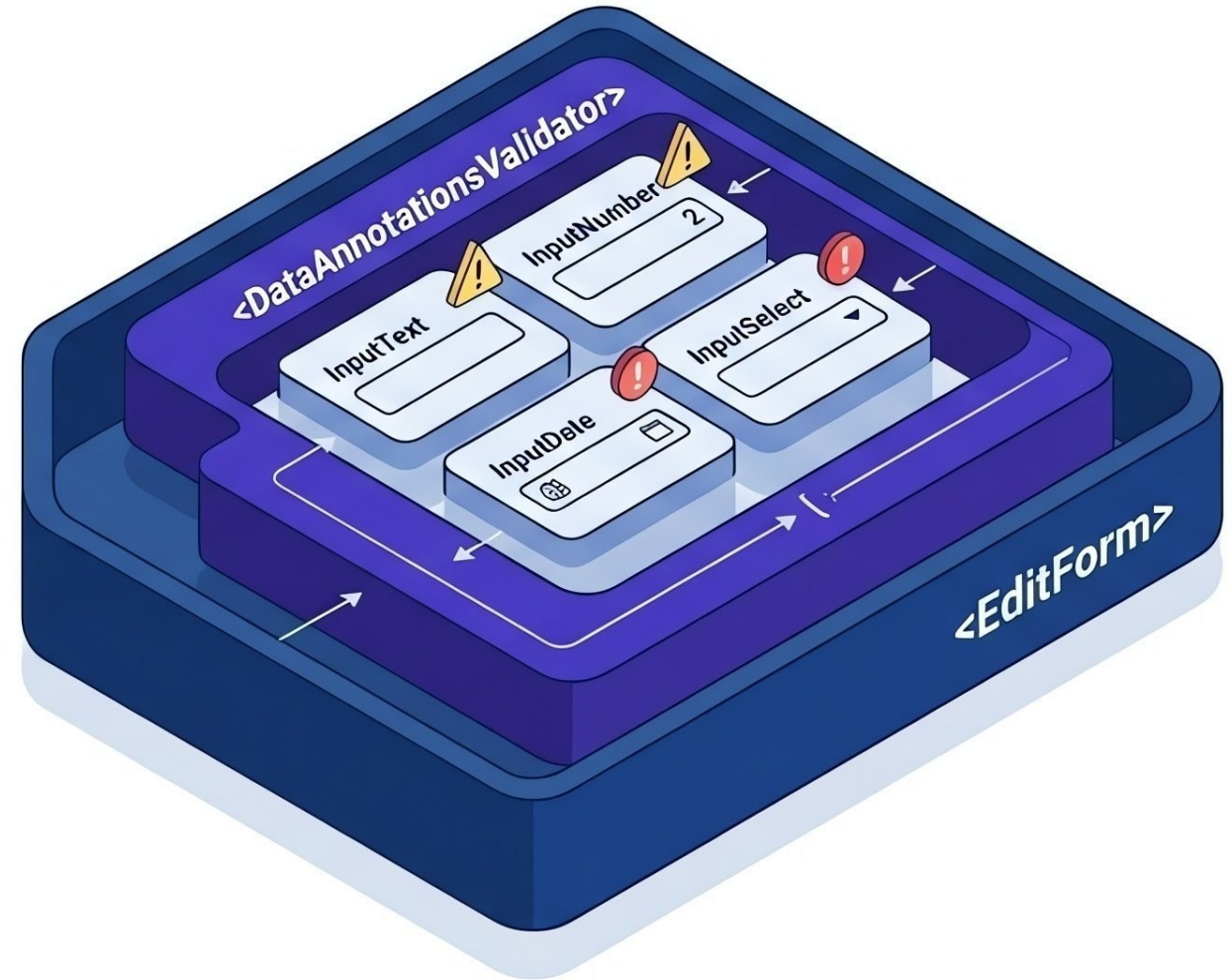
Lưu ý Kiến trúc Quan trọng: Biến @ref mang giá trị null ở giai đoạn đầu, chỉ thực sự có giá trị sau giai đoạn `OnAfterRenderAsync`.



Xử lý Biểu mẫu Chuyên sâu: EditForm & EditContext

Cấu trúc EditForm Chuẩn mực

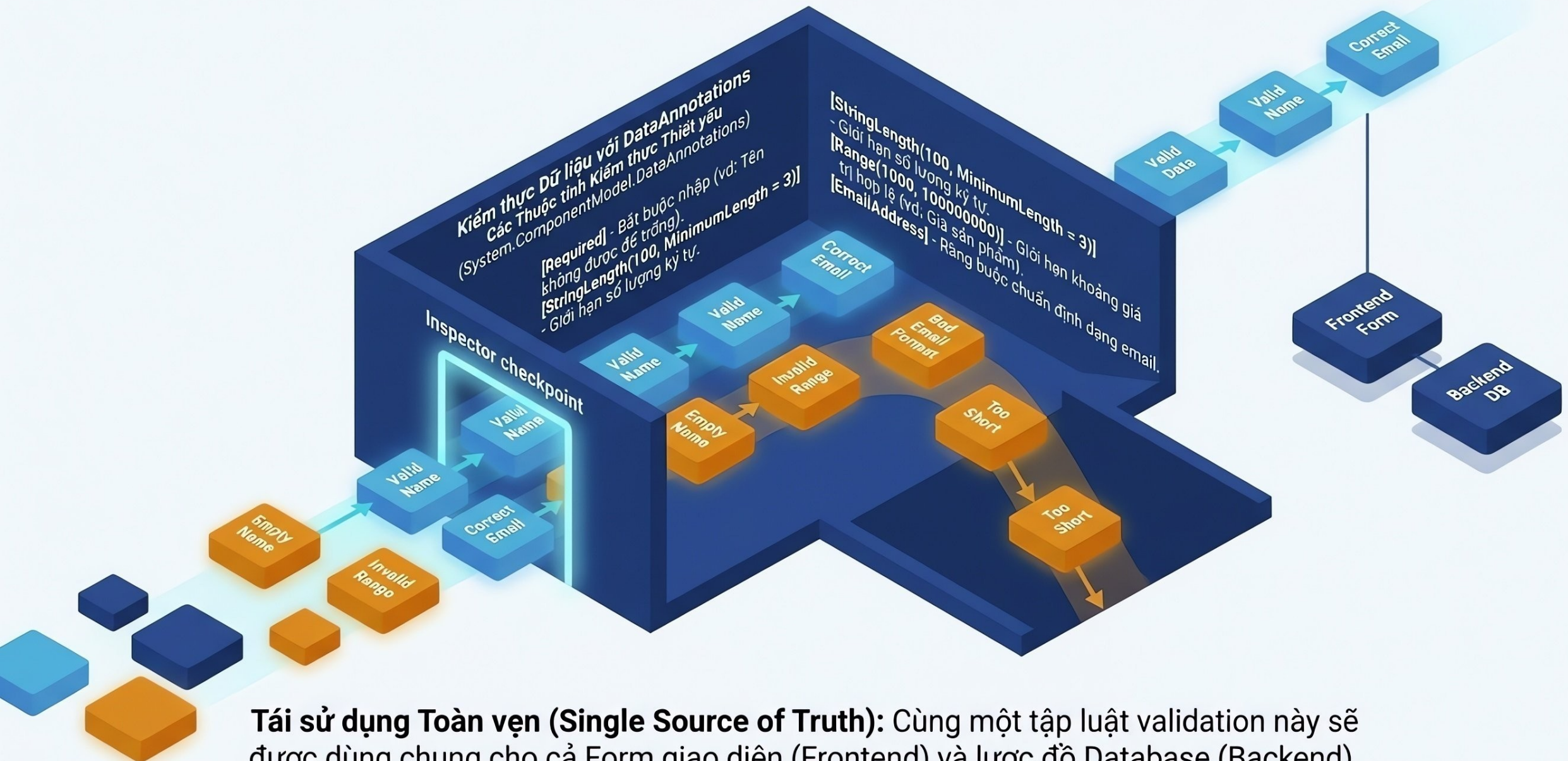
- Gắn kết đối tượng: `<EditForm Model='@order' OnValidSubmit='HandleValidSubmit'>`
- Kích hoạt bộ kiểm thực tự động: `<DataAnnotationsValidator />`
- Vùng hiển thị lỗi: `<ValidationSummary />` & `<ValidationMessage />`



Cơ chế Input Tích hợp Sẵn

- Sử dụng InputText, InputNumber, InputDate, InputSelect.
- Lợi ích: Tự động thiết lập Binding 2 chiều, đồng bộ lập tức dữ liệu người dùng gõ vào thuộc tính C#.

Kiểm thực Dữ liệu với DataAnnotations



Tái sử dụng Toàn vẹn (Single Source of Truth): Cùng một tập luật validation này sẽ được dùng chung cho cả Form giao diện (Frontend) và lược đồ Database (Backend).

Case Study Đề thi: Mổ xẻ Form Hồ sơ Cá nhân

Bài toán Thực tiễn (Bài tập 3 - Đề thi Vấn đáp): Xây dựng Form nhập hồ sơ sinh viên gồm: Họ tên, Ngày sinh, Email, Lớp, Điểm trung bình (GPA).



Ràng buộc Nghiệp vụ Kỹ thuật:

- **Họ tên:** Bắt buộc nhập ([Required]).
- **Email:** Chuẩn định dạng ([EmailAddress]).
- **GPA:** Nằm trong khoảng 0.0 đến 4.0 ([Range(0.0, 4.0)]).

Chiến lược Triển khai:

Khởi tạo Model StudentProfile gắn DataAnnotations. Bọc form bằng EditForm và xử lý lưu hồ sơ thành công tại sự kiện OnValidSubmit.

Top 5 Lỗi Thường Gặp ở Part 1 & 2 và Cách Sửa

Nhóm Lỗi Dữ liệu & Binding



Triệu chứng:
Lỗi màn hình trắng
NullPointerException /
Lỗi sự kiện.

Giải pháp:
Luôn kiểm tra `if(item != null)`
trước khi `@foreach`
Bổ sung từ khóa `await`.

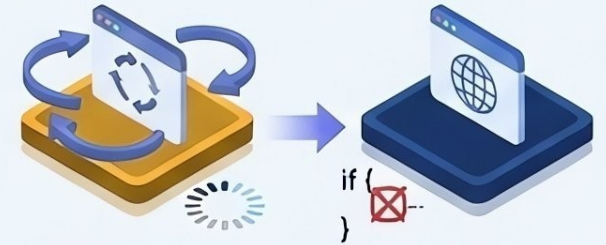


Nhóm Lỗi Vòng đời & Re-render



Triệu chứng:
Trình duyệt bị treo
(Vòng lặp vô tận).

Giải pháp:
Tuyệt đối không gọi
`StateChanged()` trong
`OnAfterRender` nếu thiếu điều
kiện chặn.

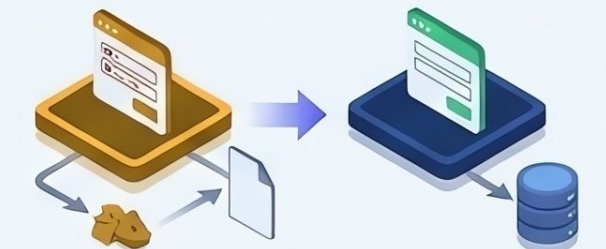


Nhóm Lỗi Biểu mẫu & Trạng thái



Triệu chứng:
Form lưu thẳng dữ liệu rác,
không báo đỏ
Mất dữ liệu khi chuyển trang.

Giải pháp:
Thêm `<DataAnnotationsValidator />`
Sử dụng State Management thay
vì lưu tạm trên giao diện.



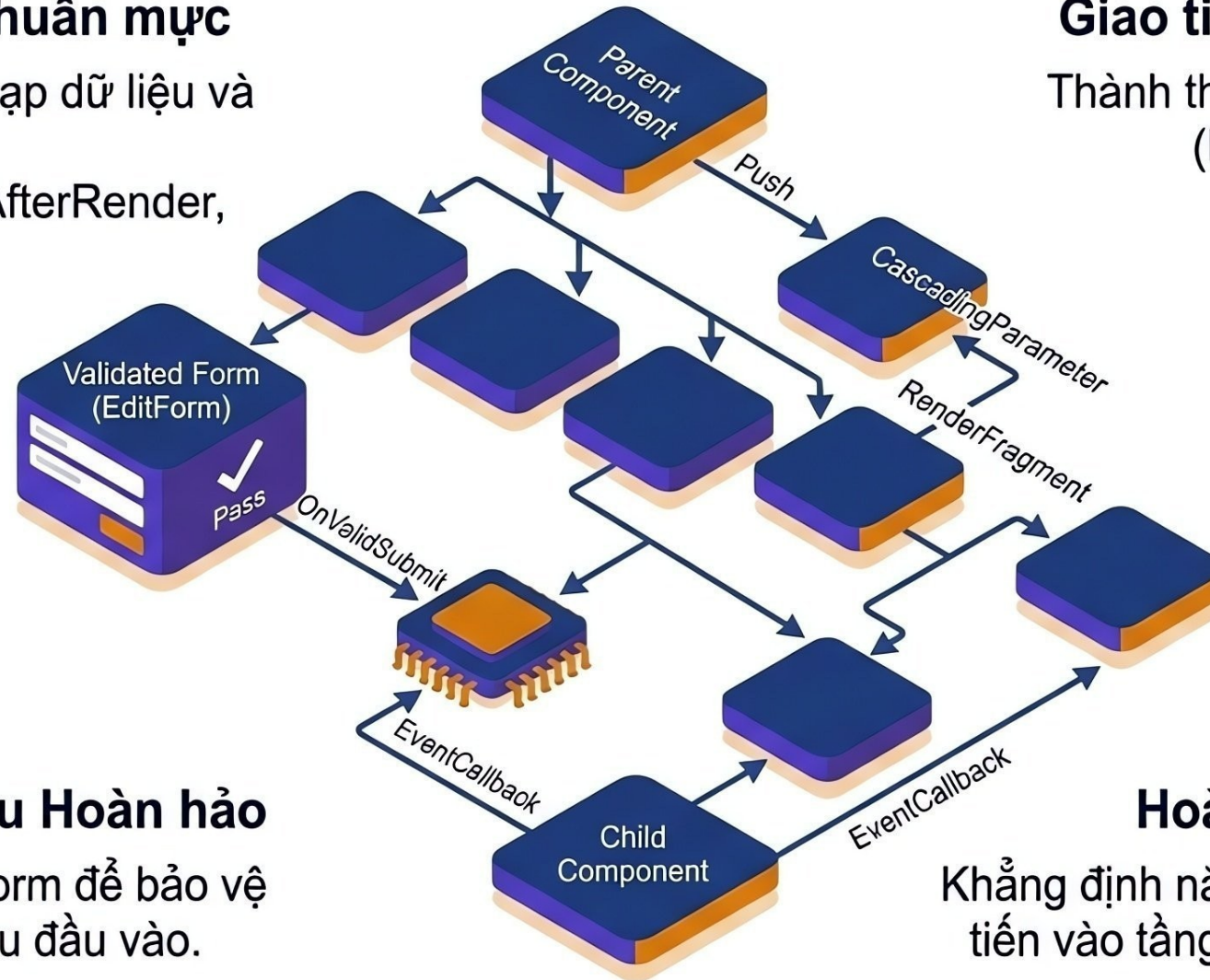
Tóm tắt Part 1: Làm chủ Hoàn toàn Kỹ thuật UI Blazor

Cơ chế Vòng đời Chuẩn mực

Đã nắm vững trình tự nạp dữ liệu và render (OnInitialized, OnParametersSet, OnAfterRender, OnAfterRender).

Giao tiếp Đa chiều Linh hoạt

Thành thạo điều phối luồng dữ liệu (Parameter, EventCallback, CascadingParameter, RenderFragment).



Kiểm thực Biểu mẫu Hoàn hảo

Làm chủ công cụ EditForm để bảo vệ tính toàn vẹn của dữ liệu đầu vào.

Hoàn thành Checkpoint 1

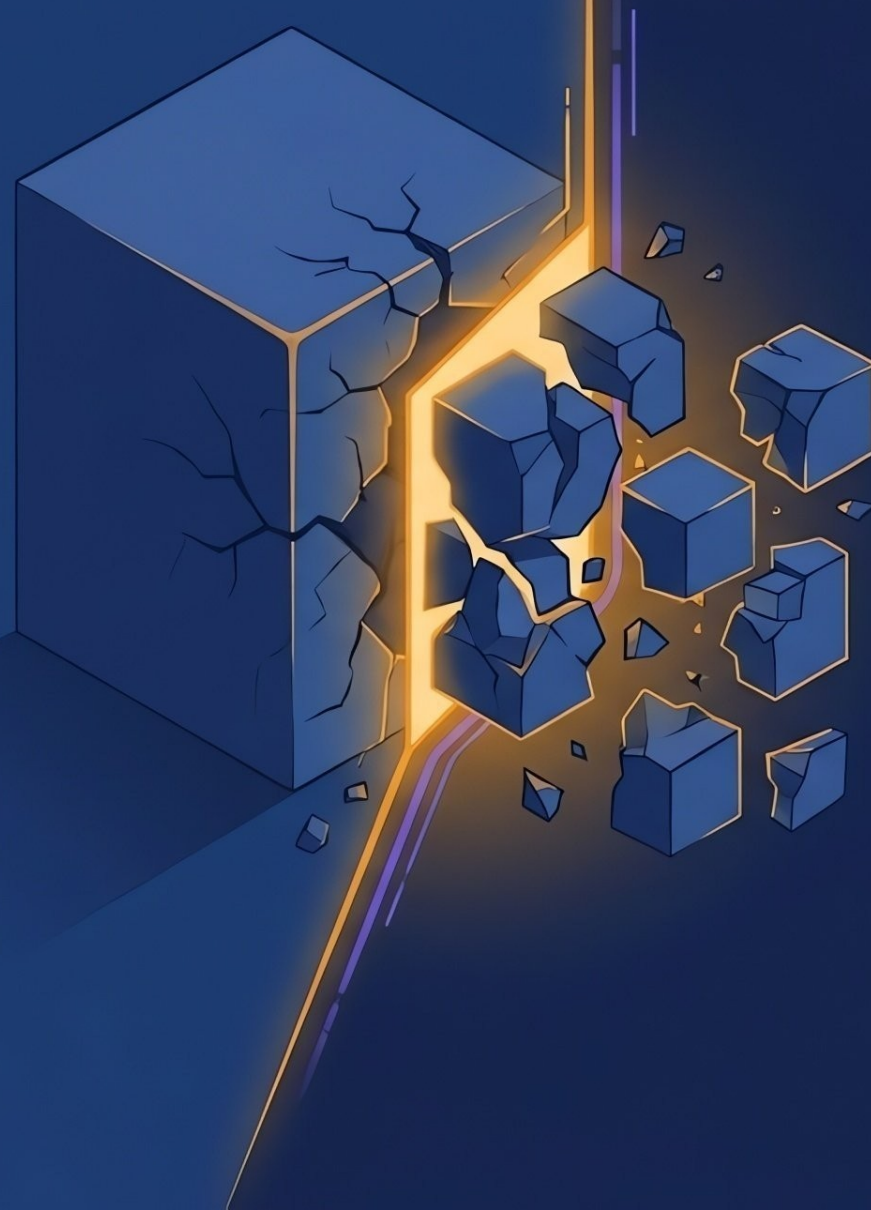
Khẳng định năng lực tự học và sẵn sàng tiến vào tầng Backend và Data Access.

Rủi ro Kiến trúc và Bước nhảy Vọt Tiếp theo

Phát hiện Bối cảnh (Điểm mù)

Hiện tại, trang **ViewProductDetails** đang gọi trực tiếp vào lớp **HardCodedProductRepository**. Mã nguồn UI đang dính chặt vào **Data**.

“
Nếu khách hàng yêu cầu kết nối SQL Server hoặc MongoDB thật ở Part 5, chúng ta có phải mở từng trang giao diện ra để sửa lại code hay không?



Bước nhảy Vọt Tiếp theo

Giải pháp Cốt lõi của Part 2:
Nguyên lý Đảo ngược Phụ thuộc (DIP) & Kiến trúc Dependency Injection!

Giao diện sẽ hoàn toàn không cần bận tâm dữ liệu đến từ đâu.



Đột phá Kiến trúc: Từ Code Dính chặt đến Clean Architecture

Mục tiêu:

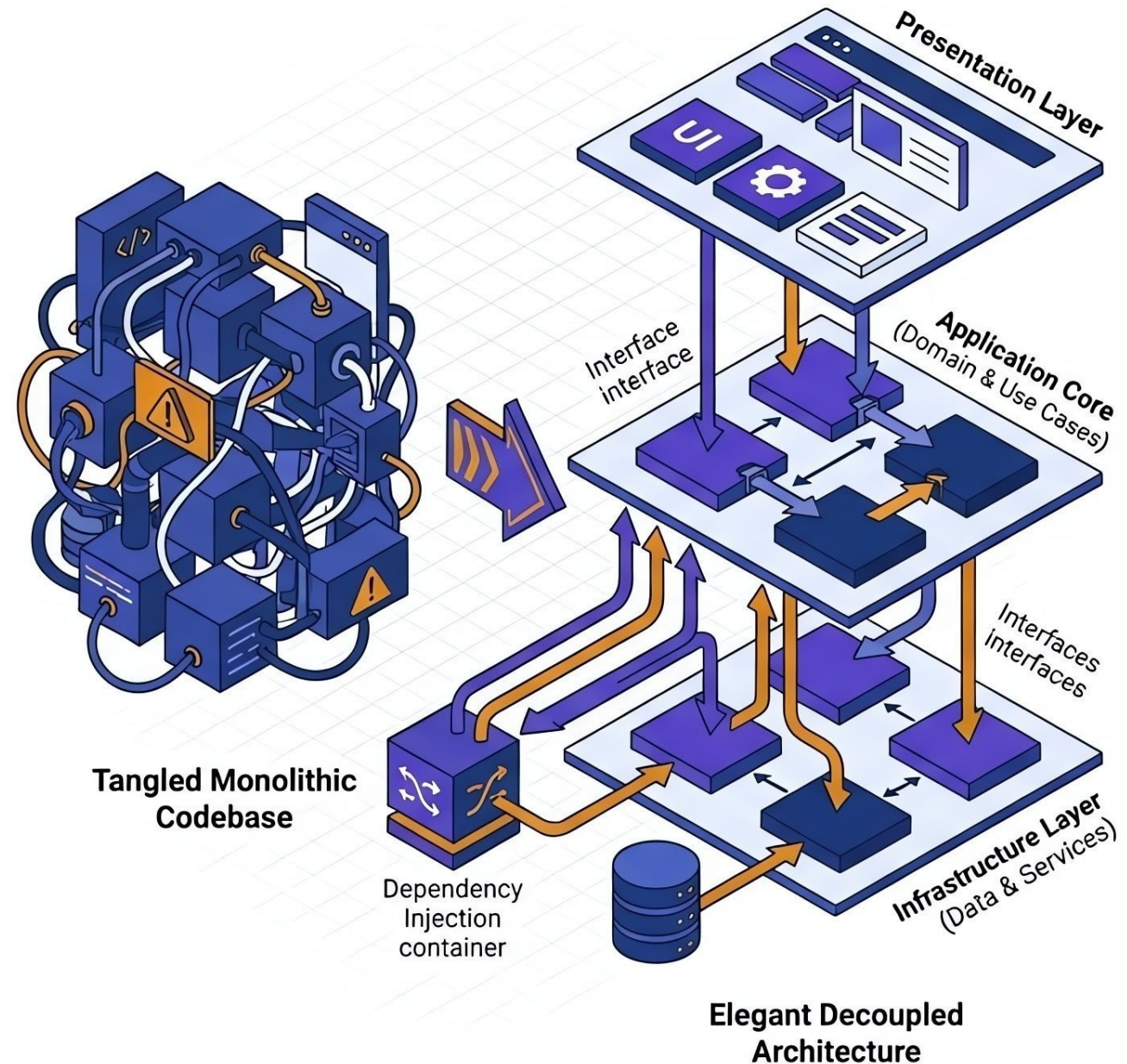
- Giải quyết triệt để bài toán phụ thuộc mã nguồn (The Dependency Problem).

Trọng tâm:

- Nắm vững DIP (Dependency Inversion Principle) & IoC (Inversion of Control).

Kỹ thuật:

- Làm chủ hệ thống Dependency Injection (DI) Container trong .NET.



The Dependency Problem: Thảm họa Tight Coupling

UI Node (Tầng Giao diện)

Trong file .razor, khởi tạo trực tiếp:
`private HardCodedProductRepository repo = new();`

Bản chất Dính chặt: Tầng Giao diện UI cấp cao bị trói chặt vào Chi tiết Kỹ thuật cấp thấp. Mọi thay đổi về Data buộc phải sửa trực tiếp file UI.

Data module
(Chi tiết Kỹ thuật cấp thấp)

Hậu quả:

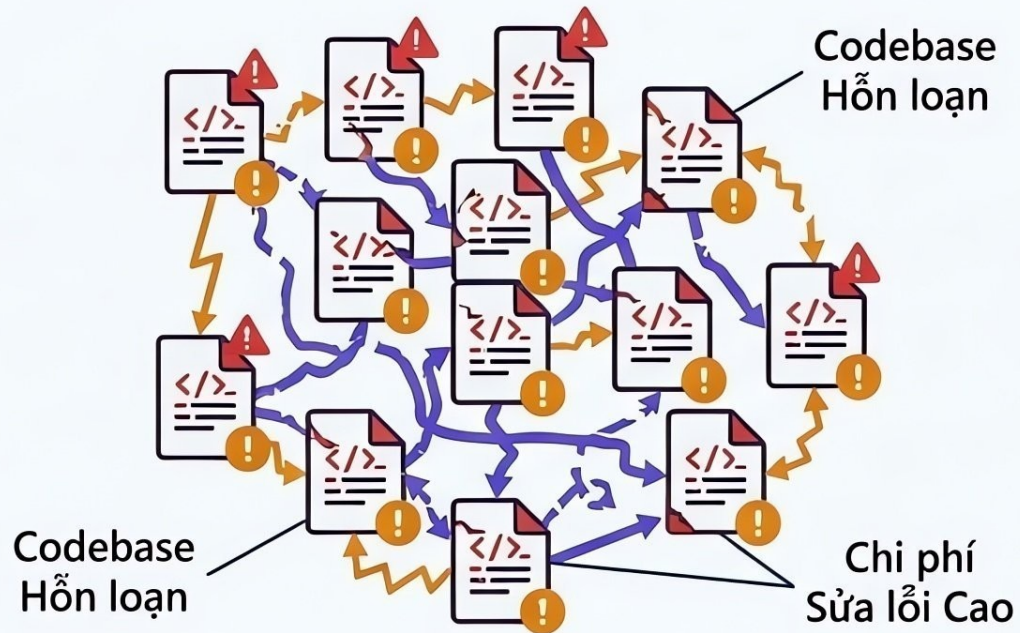
- Không thể viết **Unit Test** độc lập.
- Mã nguồn **dễ vỡ nát** khi nâng cấp.

Kịch bản Thảm họa: Khi Khách hàng Đổi Nguồn Dữ liệu

Yêu cầu: Chuyển từ In-Memory sang SQL Server với Dapper.

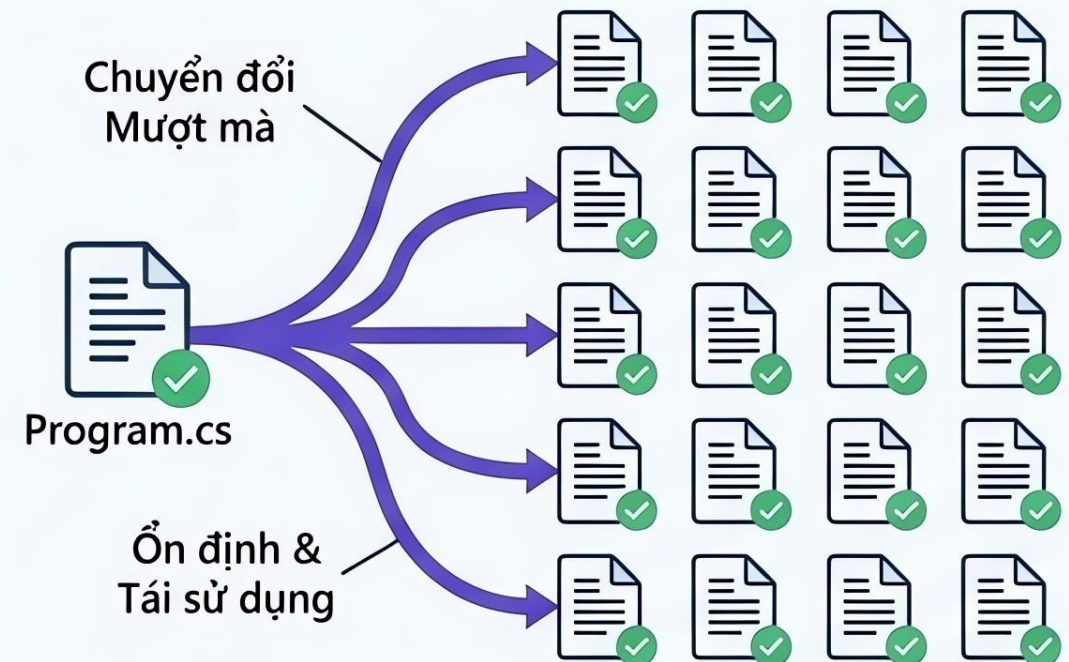
Kịch bản: Code Dính chặt

Sửa tên từ HardCoded thành SqlRepository.
15 trang Razor báo lỗi đỏ hàng loạt.
Tốn hàng ngày trời sửa lỗi thủ công.

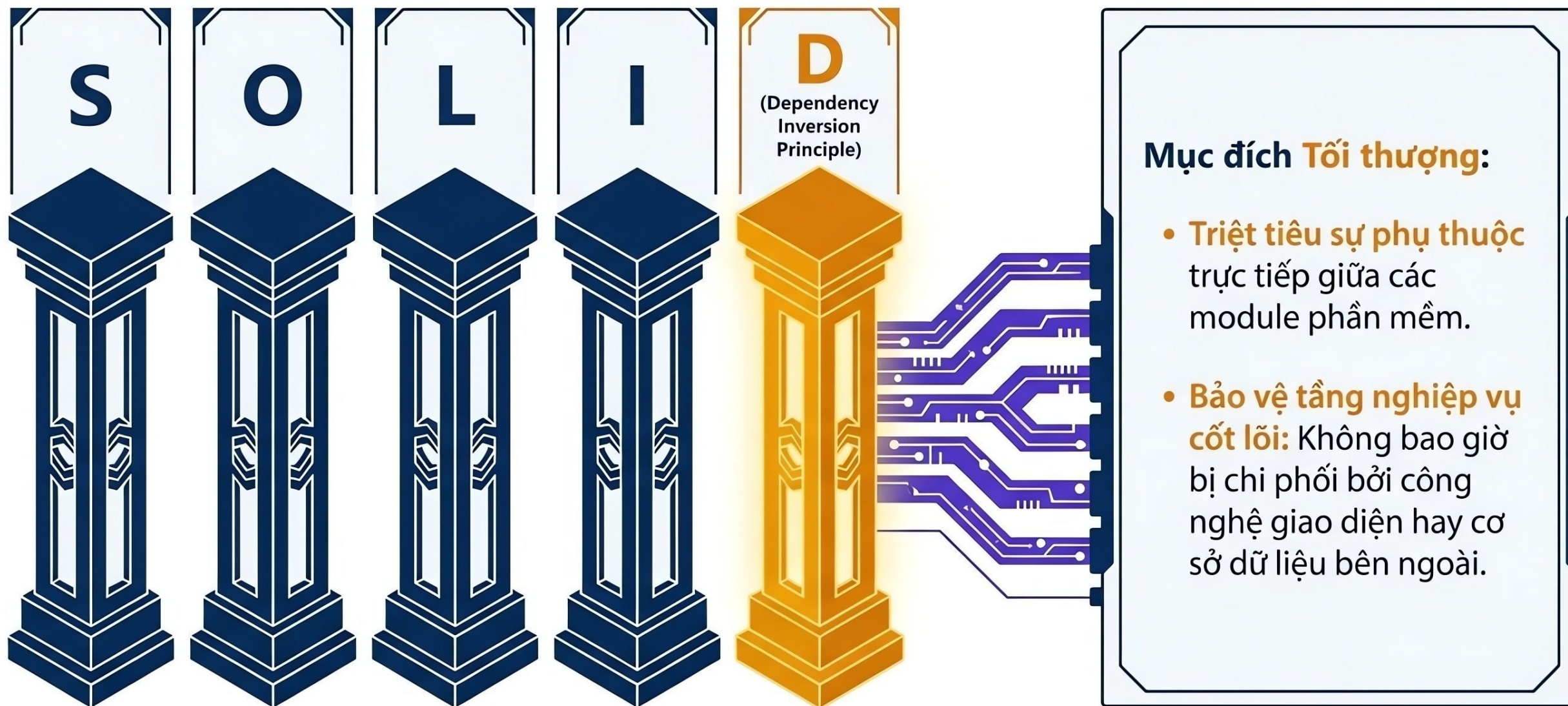


Kịch bản: Kiến trúc Sạch

Chỉ đổi đúng 1 dòng cấu hình trong Program.cs.
Toàn bộ 15 trang Razor giữ nguyên 100% không phát sinh lỗi.



Nguyên lý SOLID: Nhắc lại Chữ “D” Tối thượng



Định nghĩa Chuẩn: Dependency Inversion Principle (DIP)

Quy tắc 1 (The Reversal):

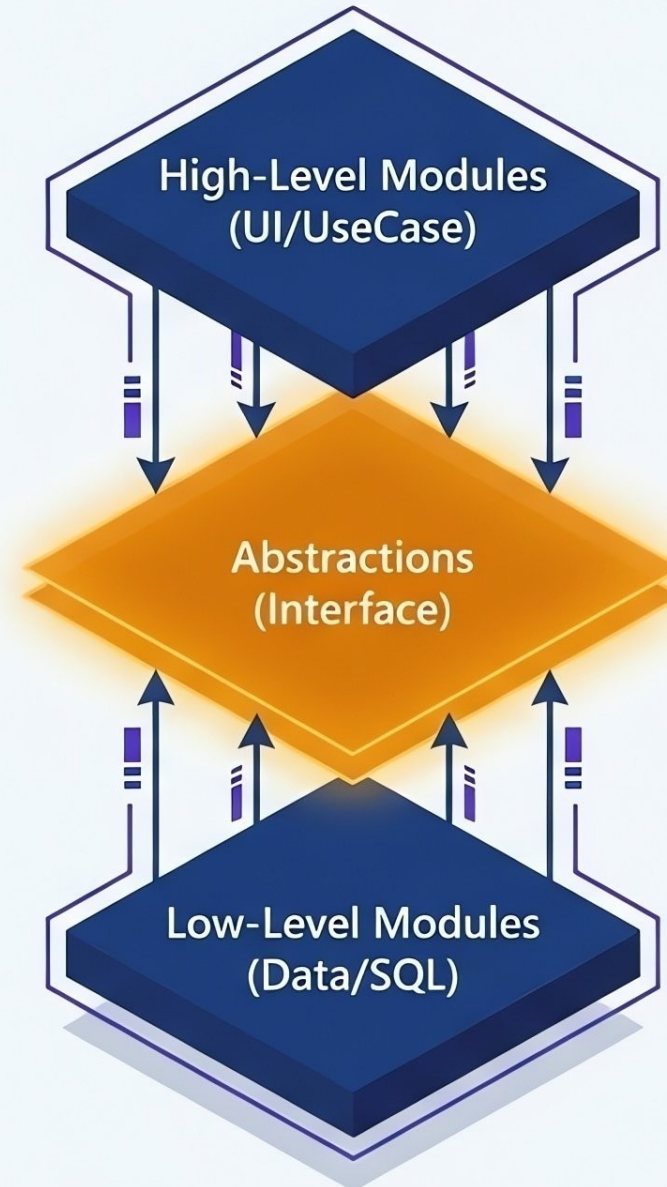
Các module cấp cao không được phụ thuộc vào các module cấp thấp. Cả hai bắt buộc phải phụ thuộc vào lớp Trừu tượng (Abstractions).

Quy tắc 2 (The Shield):

Lớp Trừu tượng không được phụ thuộc vào chi tiết cài đặt. Chi tiết cài đặt bắt buộc phải phụ thuộc vào lớp Trừu tượng.

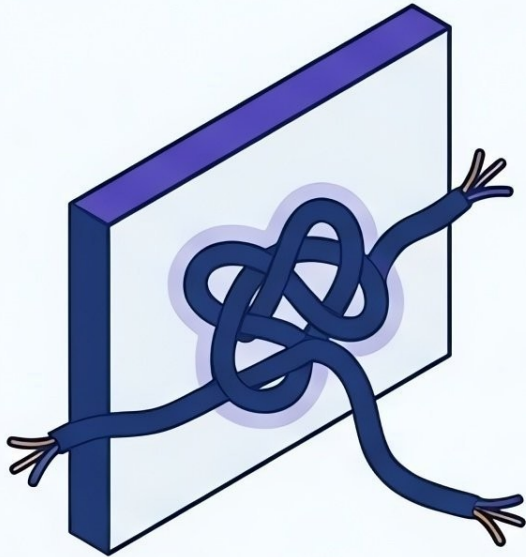
Công cụ Hiện thực hóa trong C#:

interface



Inversion of Control (IoC): Ẩn dụ Ổ cắm & Phích cắm

Mô hình Hàn chết (Code Dính chặt):



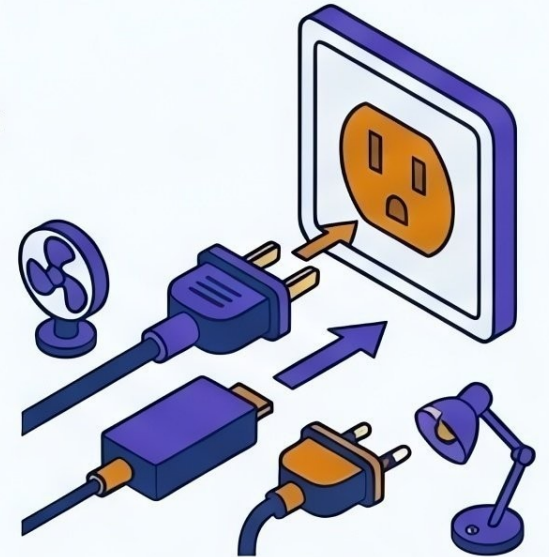
Nối thẳng dây quạt điện vào âm tường.

Muốn đổi sang máy sấy tóc -> Phải đục tường đứt mạch.

Mô hình Ổ cắm Chuẩn (IoC):

Bức tường cung cấp một ổ cắm chuẩn quốc tế (Interface).

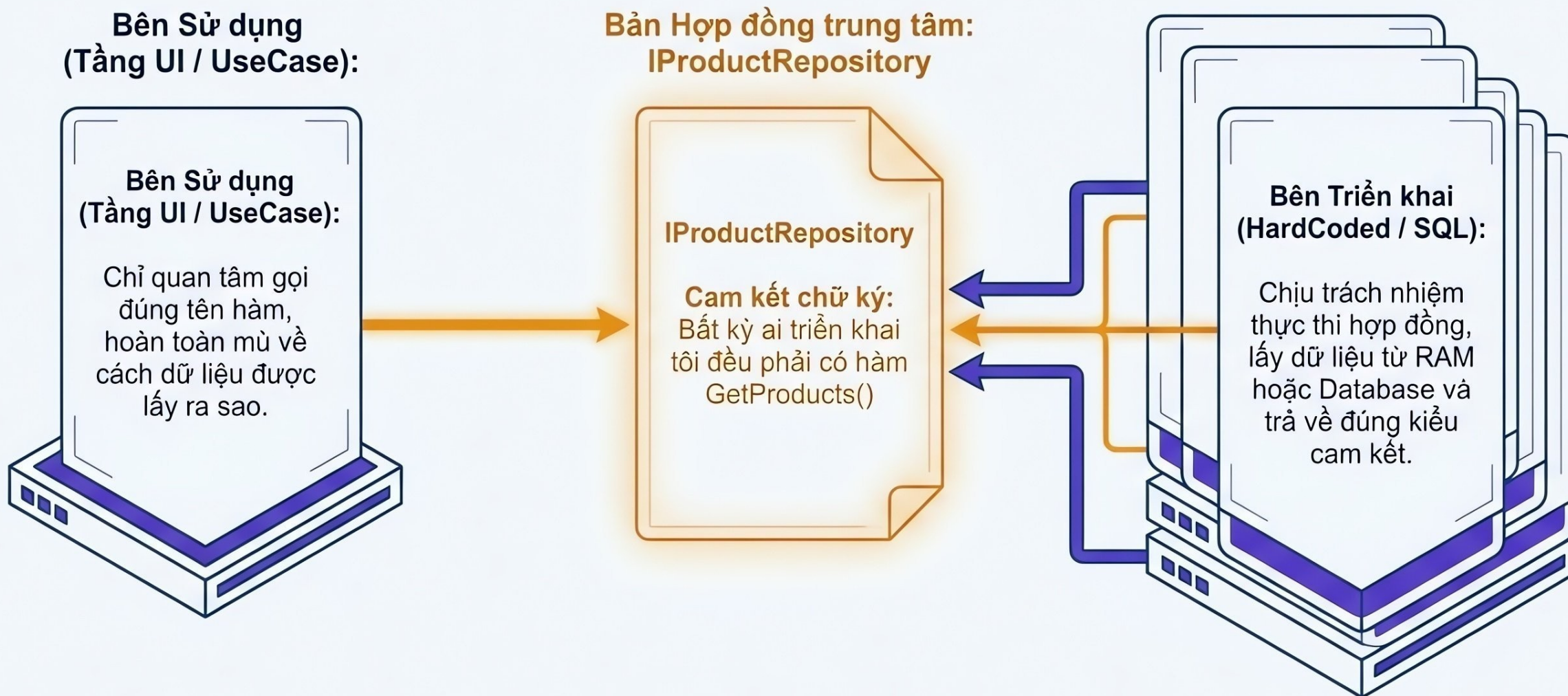
Cắm quạt, laptop hay đèn bàn tùy ý mà không cần phá vỡ cấu trúc tường.



Insight:

IoC trao quyền kiểm soát và thay thế linh kiện cho người lắp ráp hệ thống.

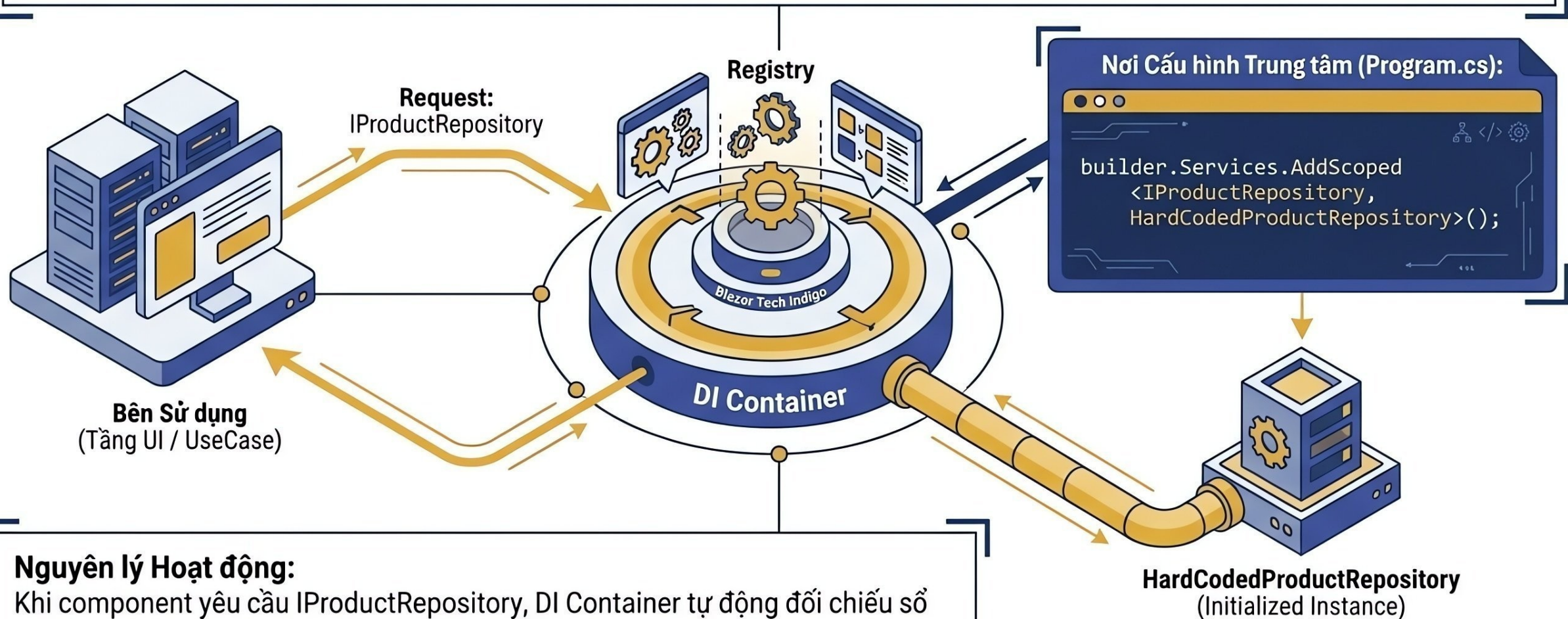
Interface là Bản Hợp đồng: Tách rời Logic khỏi Chi tiết



Dependency Injection (DI) Container trong ASP.NET Core

Vai trò cốt lõi (Nhà máy Cung cấp Phụ thuộc):

Tự động khởi tạo, quản lý vòng đời và tiêm (Inject) đối tượng vào nơi cần sử dụng mà không cần new.



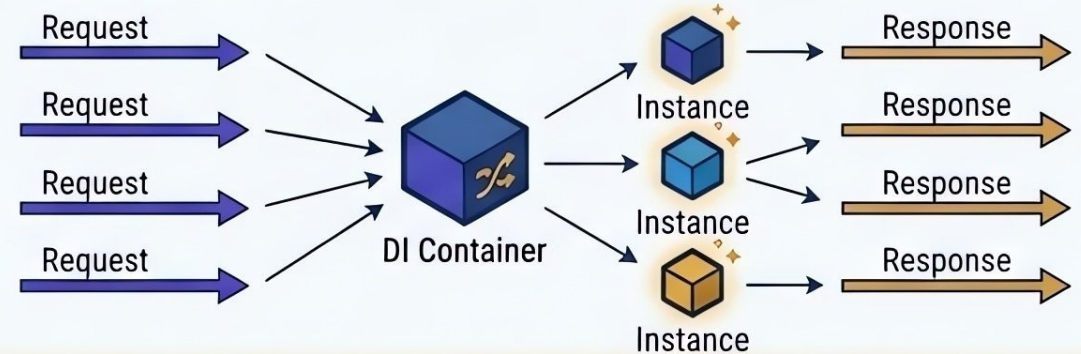
Nguyên lý Hoạt động:

Khi component yêu cầu `IProductRepository`, DI Container tự động đối chiếu số đăng ký và cung cấp chính xác module đã cài đặt.

Mổ xẻ 3 Service Lifetimes trong .NET Container

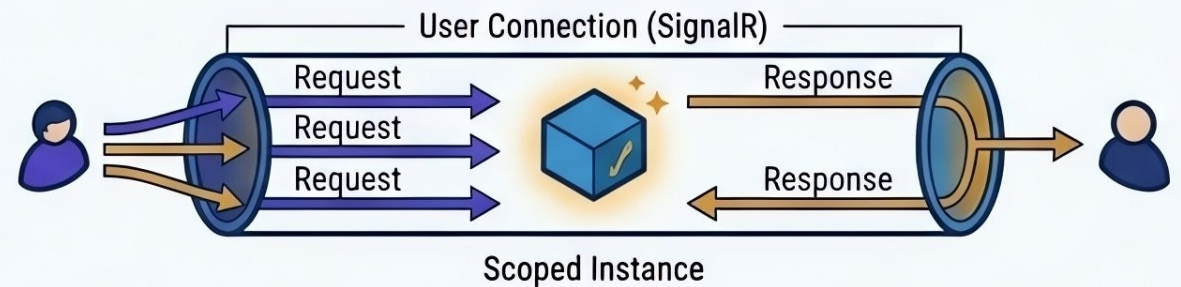
1. Transient (AddTransient):

- **Vòng đời:** Tạo mới một đối tượng riêng biệt mỗi khi có bất kỳ yêu cầu nào.
- **Thích hợp:** Dịch vụ nhẹ, xử lý nhanh, không lưu trạng thái.



2. Scoped (AddScoped - Trọng tâm Blazor):

- **Vòng đời:** Tồn tại 1 phiên bản duy nhất trong suốt vòng đời của một kết nối (SignalR).
- **Thích hợp:** Quản lý vùng nhớ riêng biệt cho từng người dùng cá nhân.



3. Singleton (AddSingleton):

- **Vòng đời:** Khởi tạo duy nhất 1 phiên bản dùng chung cho toàn bộ ứng dụng và tất cả User.

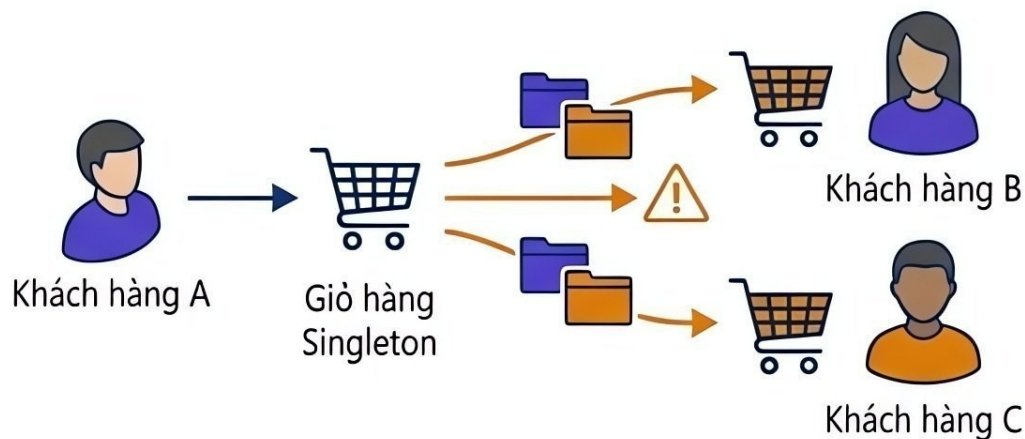


Nguy cơ Khôn lường khi Dùng Sai Lifetime trong Blazor



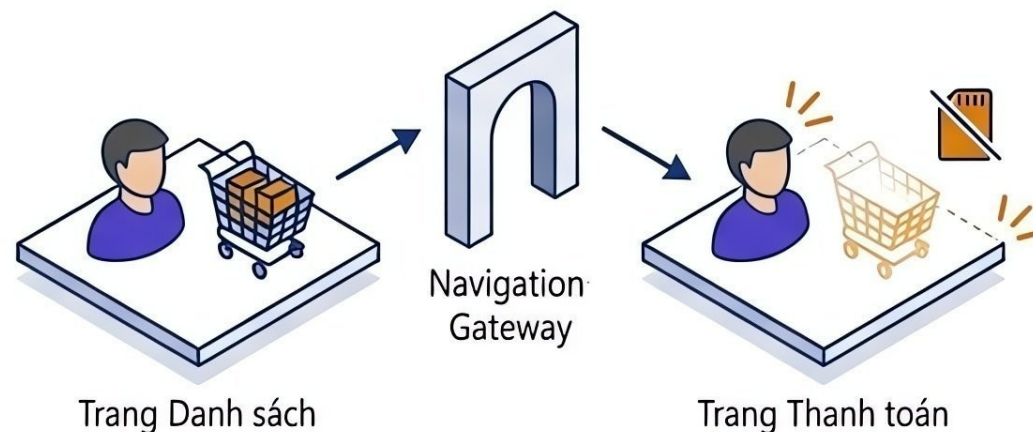
Thảm họa 1: Lạm dụng Singleton cho Giỏ hàng:

Khách hàng A thêm sản phẩm -> Tự động xuất hiện trong giỏ của Khách hàng B và C! (Rò rỉ dữ liệu chéo).



Thảm họa 2: Dùng Transient cho Quản lý Trạng thái:

Khách hàng thêm hàng ở trang Danh sách, chuyển sang trang Thanh toán thì giỏ hàng bị rỗng ngay lập tức.



Quy tắc Vàng Blazor Server:

Mọi dịch vụ lưu trạng thái cá nhân (User State, Cart, Repository) -> Bắt buộc dùng **Scoped**.



Kỹ thuật Inject Service: @inject vs Constructor

Giao diện Razor (.razor):

Cú pháp khai báo trực tiếp ở đầu trang:

```
@inject IProductRepository ProductRepository
```

```
@inject NavigationManager NavigationManager
```

Lớp C# Thuần (.cs - Constructor Injection):

Tiêm qua hàm tạo (Constructor) chuẩn mực trong các lớp nghiệp vụ UseCase.

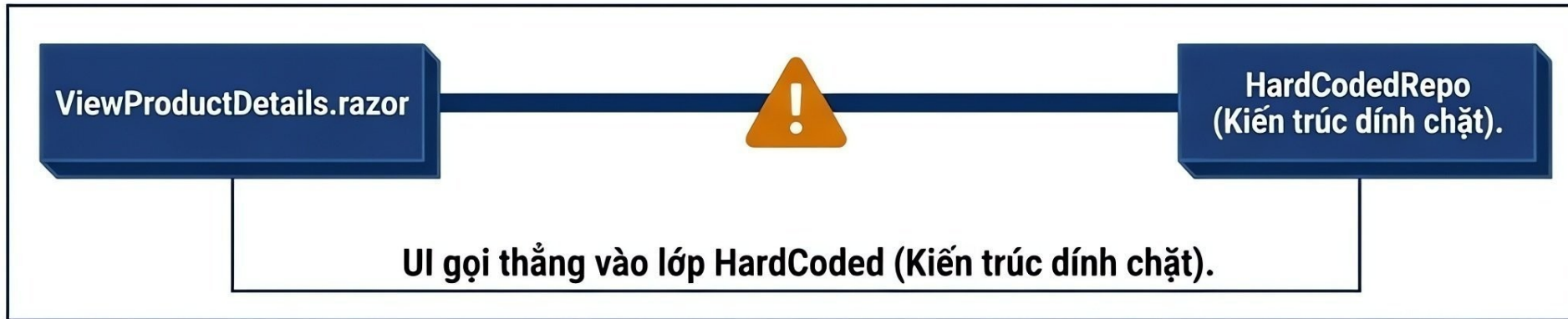
```
Constructor(inject IProductRepository,  
            Interface-NavigationManager {  
}
```



Tính Đồng nhất: Cả hai phương pháp đều nhận đối tượng được cung cấp sẵn từ DI Container, loại bỏ hoàn toàn từ khóa new.

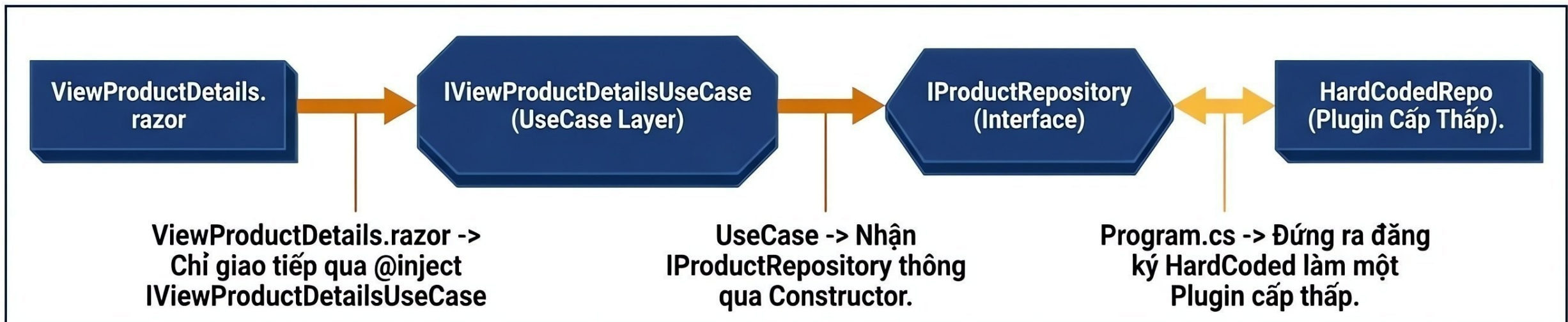
Tái cấu trúc eShop: Tách Rời Giao diện khỏi DataStore

Trạng thái Cũ (Before Refactor):



Kết quả: Giao diện hoàn toàn mù về nguồn dữ liệu thực tế!

Trạng thái Mới (Chuẩn Clean Architecture):



Đo lường Hiệu quả: Khả năng Unit Test & Thay thế Plugin

Khả năng Kiểm thử Tự động:

Dễ dàng tạo **MockProductRepository** (dữ liệu giả) để test logic giỏ hàng trong vài mili-giây mà không cần bất cứ cơ sở dữ liệu.



Thay thế Plugin trong 30 Giây:

Chuyển sang SQL Server? Chỉ cần viết lớp **SqlProductRepository** và sửa đúng 1 dòng cấu hình trong **Program.cs**.

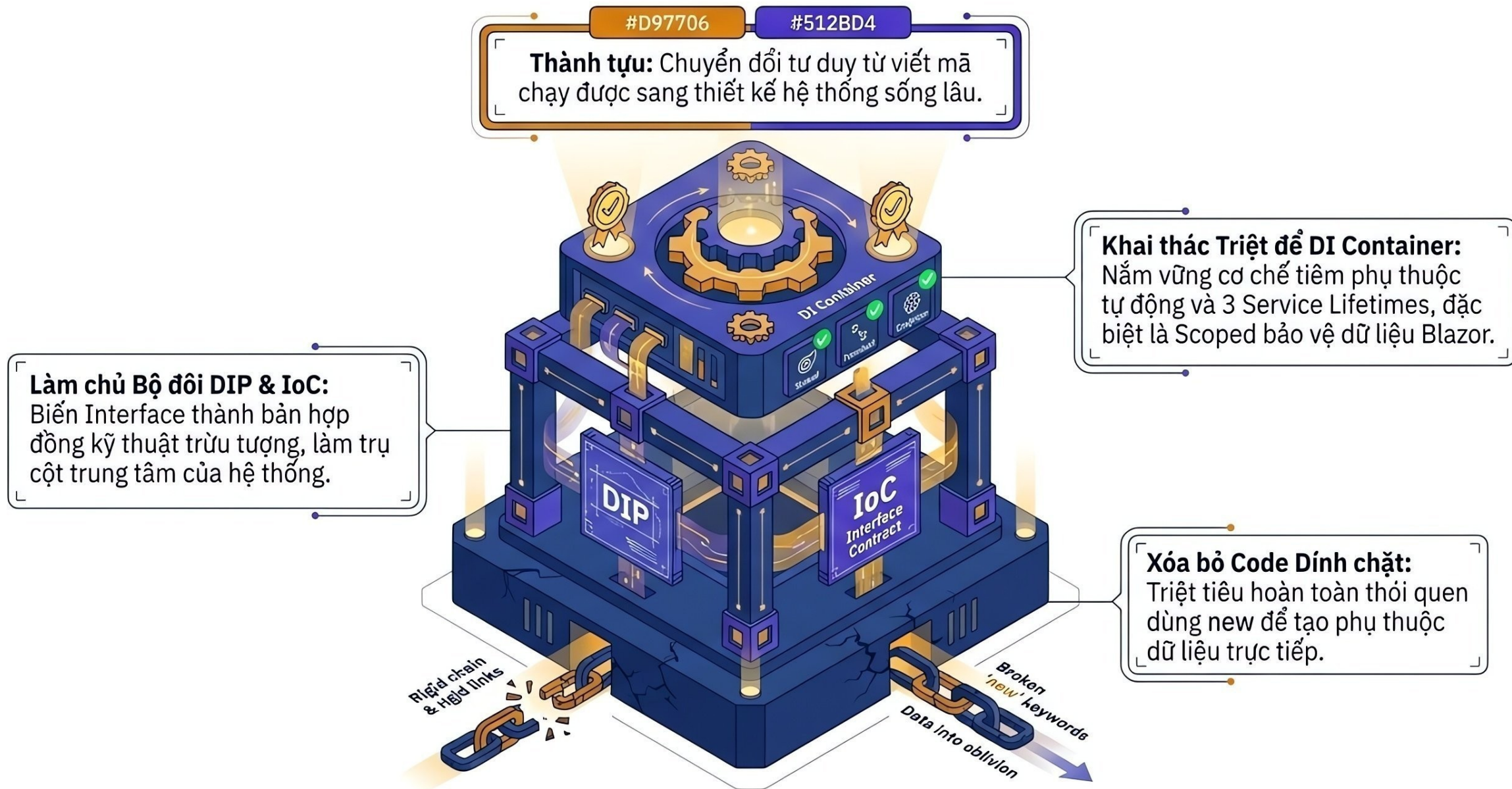


Lập trình Song song (Parallel Dev):

Đội Frontend làm giao diện độc lập với **Mock Data**, không cần chờ đội Backend xây dựng xong bảng SQL.



Tóm tắt Part 2: Khai phóng Năng lực Kiến trúc Doanh nghiệp



Cầu nối Sang Giai đoạn 2: Sẵn sàng Chinh phục Part 3

Nhiệm vụ Chuyển giao:

- Hoàn thiện bài tập **Milestone 1** và đồng bộ mã nguồn kiến trúc mới lên nhánh **Git part2-finish**.

Khai mở Chuỗi Video Part 3 (Build Customer Portal):

- Tách giải pháp thành **4 Project C#** phân tầng độc lập: **CoreBusiness, UseCase, DataStore, Web**.
- Xây dựng hoàn chỉnh luồng nghiệp vụ **Giỏ hàng & Đặt hàng** đa bước (**Checkout Pipeline**).

Mục tiêu Tối hậu:

- Nâng tầm dự án **eShop** thành sản phẩm thương mại điện tử chuyên nghiệp cấp doanh nghiệp.

